

EDUGENIE

Google Gemini Powered Learning Assistant

3. Project Design

System Architecture

- The application follows a modular web architecture.
- The user interacts with the HTML/CSS frontend. Requests are sent to the FastAPI backend, which routes each task to its corresponding AI module.
- Cloud-based Gemini services provide generative AI capabilities, while the documented lightweight local model is used for concept explanation.

Folder Architecture

- EduGenie/
- main.py – FastAPI application
- explanation_module.py – concept explanation logic
- qna.py – question answering
- quiz_module.py – quiz generation
- summary_module.py – summarization
- learning_path.py – learning recommendations
- templates/index.html – web frontend
- static/style.css – styling
- requirements.txt – Python dependencies

API Design

- /qa – handles question-answering requests.
- /explain – handles simplified concept explanations.
- /quiz – generates quiz questions.
- /summarize – summarizes supplied content.
- /learn/recommendations – produces personalized learning recommendations.

Frontend Design

- A task dropdown lets the learner select Explain, QnA, Quiz, Summary, or Recommend Path.

- A text area accepts the topic, question, passage, or other input.
- The submitted result is displayed below the input area.

Design Goals

- Keep the interface simple and responsive.
- Keep backend modules separated so that individual features can be maintained or upgraded independently.
- Provide a lightweight foundation suitable for future expansion.

Source basis: EduGenie project documentation provided by the user.